

AI-solutions — Lab 1: Modeling and Identification of the Parrot AR.Drone

Questions 2.1, 2.2, 2.3

AI-SOLUTIONS – NOT OFFICIAL. This document was generated by an AI assistant (Claude, Anthropic) as a self-check reference only. It has not been reviewed by course staff and may contain sign or convention errors. Use it exclusively to cross-check a solution worked out independently on paper, never as a submitted answer.

Setup and assumptions

We use the notation of the handout: $\mathbf{f}_T = [f_1 \ f_2 \ f_3 \ f_4]^T$ for the individual rotor thrusts, $n_i = cf_i$ for the individual reaction torques, l for the arm length (distance from the center of mass to each rotor), and $\mathbf{f}_g = mg\mathbf{e}_3$ for gravity expressed in $\{I\}$.

For Fig. 1 of the handout we take the following geometric convention, consistent with the figure:

- Rotor 1 is located on the $+x_B$ axis, rotor 4 on the $+y_B$ axis, rotor 3 on the $-x_B$ axis, rotor 2 on the $-y_B$ axis (arms aligned with the body axes, as drawn in Fig. 1 of the handout).
- Every rotor thrust points along $-z_B$ (upward, since z_B points down in the NED-aligned body frame).
- Rotors 1 and 3 spin one way and rotors 2 and 4 spin the opposite way (the two counter-rotating pairs mentioned in the statement), so that the reaction-torque contributions to yaw alternate in sign along the rotor index.

If your own numbering/sign convention differs, only the signs inside the matrices below change — the derivation method is unaffected.

2.1 – Expressions for $\mathbf{f}$ and $\mathbf{n}$

Translational force. The total external force in $\{B\}$ is the sum of gravity (expressed in $\{I\}$, rotated into $\{B\}$) and the resultant of the four rotor thrusts (already aligned with $\{B\}$, all along $-z_B$):

$$\mathbf{f} = R^T(\lambda) \mathbf{f}_g + N \mathbf{f}_T.$$

Hence

$$M = R^T(\lambda) \in SO(3), \quad N = -\mathbf{e}_3 \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & -1 & -1 & -1 \end{bmatrix} \in \mathbb{R}^{3 \times 4}.$$

(M depends on attitude because gravity is defined in $\{I\}$ and must be rotated into $\{B\}$; N is constant because every thrust is already expressed in $\{B\}$ along the same axis.)

Torque. Each rotor i produces (a) a moment from its thrust acting at moment arm l about the two horizontal axes, and (b) a reaction torque $n_i = cf_i$ about z_B , alternating in sign with the spin direction. With the geometric convention above ($\mathbf{r}_i \times (-f_i \mathbf{e}_3)$ for each rotor position $\mathbf{r}_i$):

$$n_x = l(f_2 - f_4), \quad n_y = l(f_1 - f_3), \quad n_z = c(f_1 - f_2 + f_3 - f_4),$$

so that

$$\mathbf{n} = P\mathbf{f}_T, \quad P = \begin{bmatrix} 0 & l & 0 & -l \\ l & 0 & -l & 0 \\ c & -c & c & -c \end{bmatrix} \in \mathbb{R}^{3 \times 4}.$$

2.2 – Redefining $\{B\}$ with x_B at 45°

Rotating the body axes by 45° about z_B (new x'_B midway between the arms of rotors 1 and 2) does *not* change $\mathbf{f}$: every thrust still acts along $-z_B$ (unchanged, since the rotation is about z_B itself), and $R^T(\lambda)$ is still just the attitude rotation expressed with respect to whichever axes are called $\{B\}$ – its functional form does not depend on this relabeling. Hence

M, N unchanged.

$\mathbf{n} = P\mathbf{f}_T$ *does* change, because the horizontal position of every rotor relative to the (now rotated) x_B, y_B axes changes. The new x'_B is midway between the old $+x_B$ (rotor 1's arm) and old $-y_B$ (rotor 2's arm), i.e. $x'_B = (x_B - y_B)/\sqrt{2}$, $y'_B = (x_B + y_B)/\sqrt{2}$. Expressing each rotor position in the new basis gives $\mathbf{r}'_1 = \frac{l\sqrt{2}}{2}(1, 1, 0)$, $\mathbf{r}'_2 = \frac{l\sqrt{2}}{2}(1, -1, 0)$, $\mathbf{r}'_3 = \frac{l\sqrt{2}}{2}(-1, -1, 0)$, $\mathbf{r}'_4 = \frac{l\sqrt{2}}{2}(-1, 1, 0)$ – each rotor sits at 45° from both axes, so each thrust now contributes to *both* n_x and n_y (instead of purely one, as before). Using $\mathbf{r} \times (-f\mathbf{e}_3) = (-fr_y, fr_x, 0)$ for each rotor:

$$n_x = \frac{l\sqrt{2}}{2}(f_2 + f_3 - f_1 - f_4), \quad n_y = \frac{l\sqrt{2}}{2}(f_1 + f_2 - f_3 - f_4),$$

while n_z is unchanged (yaw reaction torque depends only on spin direction, not on the axes' in-plane orientation):

$$n_z = c(f_1 - f_2 + f_3 - f_4).$$

So

$$P' = \begin{bmatrix} -\frac{l\sqrt{2}}{2} & \frac{l\sqrt{2}}{2} & \frac{l\sqrt{2}}{2} & -\frac{l\sqrt{2}}{2} \\ \frac{l\sqrt{2}}{2} & \frac{l\sqrt{2}}{2} & -\frac{l\sqrt{2}}{2} & -\frac{l\sqrt{2}}{2} \\ c & -c & c & -c \end{bmatrix} \neq P.$$

This is exactly the standard “+ configuration vs. $\times$ configuration” distinction: in the + configuration (2.1) each rotor loads a single rotational axis; in the $\times$ configuration (2.2, the real AR.Drone) every rotor loads both horizontal axes simultaneously, with a $\sqrt{2}/2$ geometric factor.

2.3 – Input transformation $\mathbf{u} = L\mathbf{f}_T$

Stacking $T = \sum_i f_i$ (from 2.1, first row all ones) on top of $\mathbf{n} = P\mathbf{f}_T$ (from 2.1) gives

$$\mathbf{u} = \begin{bmatrix} T \\ \mathbf{n} \end{bmatrix} = L\mathbf{f}_T, \quad L = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & l & 0 & -l \\ l & 0 & -l & 0 \\ c & -c & c & -c \end{bmatrix}.$$

Usefulness. L is a constant, invertible matrix ($\det L \neq 0$ for $l, c \neq 0$), so the map $\mathbf{f}_T \leftrightarrow \mathbf{u}$ is a lossless linear change of variables. Its point is that the rigid-body dynamics (1)–(3) only ever depend on $\mathbf{f}_T$ through exactly these four combinations: T enters the translational dynamics (scaled along z_B) and $\mathbf{n}$ enters the rotational dynamics directly. The individual f_i are physically coupled (each one affects both T and one or two components of $\mathbf{n}$); $\mathbf{u}$ decouples the problem into four independent virtual channels – one for altitude/thrust and three for the body-frame moments (roll, pitch, yaw). This lets the height and attitude controllers be designed directly in terms of T and $\mathbf{n}$ (as done throughout the rest of the guide), with the physical per-rotor commands recovered only at the very last step via L^{-1} – i.e., L is exactly the control-allocation / mixer stage of a real flight controller. The only caveat is that not every $\mathbf{u}$ is achievable, since $\mathbf{f}_T$ must remain nonnegative and bounded.

Inverse transformation. Solving $L\mathbf{f}_T = \mathbf{u}$ directly (summing/subtracting pairs of rows):

$$\begin{aligned} f_1 &= \frac{T}{4} + \frac{n_y}{2l} + \frac{n_z}{4c}, & f_2 &= \frac{T}{4} + \frac{n_x}{2l} - \frac{n_z}{4c}, \\ f_3 &= \frac{T}{4} - \frac{n_y}{2l} + \frac{n_z}{4c}, & f_4 &= \frac{T}{4} - \frac{n_x}{2l} - \frac{n_z}{4c}, \end{aligned}$$

i.e. $\mathbf{f}_T = L^{-1}\mathbf{u}$ with

$$L^{-1} = \begin{bmatrix} 1/4 & 0 & 1/(2l) & 1/(4c) \\ 1/4 & 1/(2l) & 0 & -1/(4c) \\ 1/4 & 0 & -1/(2l) & 1/(4c) \\ 1/4 & -1/(2l) & 0 & -1/(4c) \end{bmatrix}.$$

Each rotor receives a quarter of the total thrust plus/minus the roll, pitch and yaw contributions needed to produce the requested $\mathbf{n}$ – the familiar quadrotor mixer formula.